과제 2

독서 기록 REST API 서버

Express Router · Controller · 미들웨어 통합 과제

소스 + README 제출

과제 개요

- **주제** — 도서 정보를 JSON 파일에 저장하고 등록 · 조회 · 수정 · 삭제 · 통계 기능을 제공하는 독서 기록 REST API 서버를 Express 5 + TypeScript로 구현
- **통합** — Router · Controller · Store 역할 분리, 기본 미들웨어와 추가 미들웨어, 비동기 파일 입출력을 하나의 서버로 결합
- **제출물** — 완성한 프로젝트 폴더 전체(node_modules 제외)와 이해 내용을 기술한 README.md를 함께 제출
- **평가** — 코드 실행 70% · README 20% · 코드 품질 10%

과제 내용 — 구현할 서버의 기능

- **서버의 역할** — 읽고 싶은 · 읽는 중인 · 다 읽은 도서를 저장하고 조회 · 등록 · 수정 · 삭제하는 독서 기록 API 서버 (data/books.json 파일에 저장)

- **도서 정보** — 도서마다 다음 값을 저장
  - **id · title · author** — 도서 번호(서버가 부여) · 제목 · 저자
  - **status** — 읽기 상태: want(읽고 싶음) · reading(읽는 중) · done(완독)
  - **rating** — 별점: 1~5 정수, 평가하지 않았으면 null
  - **memo · createdAt** — 메모 · 등록 시각(서버가 기록)

- **조회** — 읽기 상태(status)와 검색어(keyword: 제목 · 저자)로 목록 조회 · 도서 한 권 조회 · 상태별 도서 수 조회
- **등록** — 제목·저자는 필수, 나머지는 생략 시 기본값, 같은 제목 + 저자는 중복 등록 불가, 별점은 완독 도서만 입력
- **수정** — 읽기 상태·별점·메모 중 원하는 값만 수정, 완독에서 다른 상태로 바뀌면 별점 삭제
- **삭제** — 등록한 도서를 id로 삭제

구현 요구 — API 명세

요청	성공 응답	실패 응답
`GET /books`	200 + 도서 배열	400 잘못된 status
`GET /books/stats`	200 + { want, reading, done, total }	—
`GET /books/:id`	200 + 도서	400 잘못된 id · 404 없음
`POST /books`	201 + 저장된 도서	400 · 409 중복
`PATCH /books/:id`	200 + 수정된 도서	400 · 404
`DELETE /books/:id`	204 (본문 없음)	400 · 404
그 외 경로	—	404

구현 요구 — 목록 조회 GET /books

query	허용 값	생략 시
`status`	'want' · 'reading' · 'done'	조건 없음
`keyword`	제목 또는 저자에 포함된 문자열 (영문 대소문자 구분 없음)	조건 없음

```
GET /books?status=done&keyword=김
→ 200  도서 3권 배열 (살인자의 기억법 외 2권)

GET /books?keyword=code
→ 200  도서 1권 배열 — Clean Code

GET /books?status=finish
→ 400  { "message": "…" }
```

- **처리 순서**
  1. status · keyword 읽기
  2. 잘못된 status → 400
  3. 같은 status의 도서만 남김
  4. 제목 · 저자에 keyword 포함
  5. 남은 도서 배열로 응답

구현 요구 — 등록 · 수정 규칙

- **도서 필드** — id · title · author · status · rating · memo · createdAt
- **등록: POST /books**
  - **title · author 필수** — 앞뒤 공백 제거 후 빈 문자열이면 400
  - **기본값** — status는 'want', memo는 ''(빈 문자열), rating은 null, id는 기존 최댓값 + 1
  - **중복** — 같은 제목 + 저자의 도서가 이미 있으면 409
  - **별점 rating** — 1~5 정수, 등록 · 수정 후의 상태가 완독(done)인 도서만 가능 · 위반 시 400
- **수정 PATCH /books/:id** — status · rating · memo 중 전달된 필드만 변경
  - **done → 다른 상태 :** rating을 null로 초기화

역할 분리 상태 코드는 미들웨어와 Controller가 결정. Store는 데이터 · undefined · boolean만 반환

구현 요구 — 미들웨어

- **기본 미들웨어** — app.ts에 morgan · express.static(public) · express.json() 장착
- **추가 미들웨어 (필수 구현)**
  - **validateId** — :id가 1 이상의 정수가 아니면 400 · :id가 있는 endpoint에 적용
  - **requestTimer** — 응답 완료 시 처리 시간 출력 · bookRouter 전체에 적용
  - **errorHandler** — 잘못된 JSON 본문은 400 · 그 외 오류는 500 · { message } 형식
- **404 처리** — notFound는 제공 · 어떤 Router도 응답하지 않은 요청을 처리

로그 형식 : [time] GET /books?keyword=code 200 3ms

/books 이외의 요청(GET /users 등)은 로그를 기록하지 않음

구현 요구 — 파일 · 함수 명세 ①

파일	TODO · 함수	하는 일
app.ts	1	공통 미들웨어 → Router 장착 → 404 → 에러 처리
middlewares/validateId.ts	2 validateId	id가 1 이상의 정수가 아니면 400
middlewares/requestTimer.ts	3 requestTimer	응답 완료 시 처리 시간을 지정 형식으로 출력
middlewares/errorHandlers.ts	4 errorHandler	잘못된 JSON 본문은 400, 그 밖의 오류는 500
routes/bookRouter.ts	5	endpoint 6개를 Controller와 연결, 미들웨어 장착

제공 파일 types.ts · jsonFile.ts · public/index.html · notFound는 수정하지 않음

구현 요구 — 파일 · 함수 명세 ②

파일	TODO · 함수	하는 일
store.ts	6 findAll	status · keyword 조건에 맞는 도서 배열 반환
	7 exists	같은 제목 + 저자의 도서 존재 여부 반환
	8 create	기본값과 id를 채워 저장하고, 저장한 도서 반환
	9 update	전달된 필드만 변경, 완독 해제 시 별점 초기화
	10 remove	삭제 후 성공 여부 반환
	11 countByStatus	상태별 도서 수 · 전체 도서 수 반환
controllers/bookController.ts	12~16	list · create · update · remove · stats

제공 예시 store.findById · bookController.getById — 비동기 Store 함수와 Controller 함수의 형태 참고

실행 · 확인 방법과 배점

```
$ npm install
$ npm run check     # 타입 검사
$ npm run dev       # http://localhost:3000
$ npm run reset     # 데이터 초기화

requests.http — Send Request
### 4. 검색어 → 200 · 1권 (Clean Code)
GET {{baseUrl}}/books?keyword=code

콘솔
[time] GET /books?keyword=code 200 2ms
[time] PATCH /books/3 400 1ms
```

- **코드 실행 70%**
  - check 오류 0개
  - requests.http 기대 결과와 일치
- **README 20%**
- **코드 품질 10%**
  - 지정 주석 3곳

제출물 — 이해했음을 증명하기

- README.md (필수)
  - **구조 · 동작 설명** — 질문 Q1~Q3에 근거 코드를 들어 답함
  - **오류 기록** — 오류 메시지 원문과 해결 과정을 최소 2건 기록함
  - **AI 활용 보고** — 사용 부분과 방식을 정직하게 기술함 (사용 허용, 이해 전제)
  - **테스트 시나리오** — requests.http에 제공되지 않은 요청 3개 이상과 기대 결과를 기술함

- **소스코드에 지정 주석 3곳**
  - /stats 선언 순서 · Controller가 별점 규칙을 검사하는 이유 · create의 await 저장 근거를 기술
  - **// [설명]** 로 남김

시작하기와 제출 체크리스트

```
과제2-reading-log/
  package.json · package-lock.json
  tsconfig.json · .env
  data/books.seed.json · books.json
  public/index.html    (제공)
  requests.http · README.md
  src/
    app.ts              <- TODO 1
    middlewares/        <- TODO 2~4
    routes/             <- TODO 5
    store.ts            <- TODO 6~11
    controllers/        <- TODO 12~16
    types.ts · jsonFile.ts  (제공)

$ npm install
$ npm run check
$ npm run dev
```

- 스타터 압축을 풀고 npm install로 의존성을 설치
- src의 TODO를 번호 순서대로 구현함
- requests.http로 결과를 확인하고 추가 시나리오를 3개 이상 작성
- README.md의 모든 항목을 작성
- node_modules를 제외하고 프로젝트 폴더 전체를 압축해 제출

평가 기준 (30점 만점)

영역	항목	배점
A. 코드 실행 (21)	컴파일 (npm run check 오류 0개)	1
	공통 미들웨어 장착 (하)	2
	추가 미들웨어 — validateId (중하)	2
	추가 미들웨어 — requestTimer (중)	2
	목록 조회 — status · keyword (중)	3
	단건 조회 · 삭제 (하)	2
	등록 — 기본값 · 400 · 409 (중)	3
	수정 — 부분 수정 · 별점 규칙 (상)	3
	통계 — 집계 · 경로 순서 (중하)	2
	파일 저장 — 재시작 후 유지 (중)	1
B. README (6)	구조 · 동작 설명 (Q1~Q3)	3
	오류 기록	1
	AI 활용 보고	1
	테스트 시나리오 설명	1
C. 코드 품질 (3)	지정 주석 3곳	3
D. 제출기준미달-감점	node_modules 포함 제출 또는 필수 파일 미제출(프로젝트 폴더내 모든 파일)	-1
F. 지연 제출/재제출	제출기한 이후 제출, 제출 파일 오류로 평가 후 재제출	-3